# אלגוריתמים 2 - הרצאה 5

14 בנובמבר 2026

**גיא יער־און**

סיכום זה נכתב במהלך סמס א' תשפ"ז (2027) במהלך הרצאותיהם של פרופסור אלי פורת ופרופסור משה לויינשטיין.
השימוש בסיכום באחריותכם בלבד.

## 1 אלגוריתמים פרמטריים (FPT)

**הגדרה 1.** אלגוריתמים פרמטריים הם דרך להתמודד עם בעיות NP־קשות. מדובר על אלגוריתם אשר זמן הריצה שלו הוא מהצורה $f(k)\cdot n^{\mathcal{O}(1)}=\mathcal{O}^*(f(k))$, כך ש$k$ הוא פרמטר כלשהו המקיים $k<<n$ . בעיה נקראת "ניתנת לפתרון ביעילות ביחס לפרמטר קבוע" ונאמר כי היא $FPT$ אם קיים עבורה אלגוריתם פרמטרי.

הערה 2. נסביר בהרחבה על מהו אלגוריתם פרמטרי. נניח אלגו' שזמן ריצתו $\binom{n}{k}$ - זהו לא אלגוריתם פרמטרי! זהו לא אלגוריתם מהצורה לעיל. עם זאת, בעיה שקיים לה אלגוריתם פרמטרי היא כזו שהתלות בגודל הקלט פולינומית בלבד, והחלק הקשה מוכל בפרמטר. למשל $2^k\cdot n$ זהו זמן ריצה טוב לאלגוריתם פרמטרי.

### 1.1 K-Path

**הגדרה 3.** בעיית K-path מוגדרת באופן הבא:
קלט - גרף מכוון $G$ ופרמטר $k$.
פלט - 1 אמ"מ קיים ב$G$ מסלול פשוט ( ללא חזרה על קודקודים) באורך לפחות $k$ צמתים.
אזי, $K-\text{Path}$ היא $FPT$.

הערה 4. נשים לב כי K-path היא בעיה $NPC$. אם נקח $k=n$ אזי זו בעיית "המסלול ההמילטוני", שהיא כמובן ב$NPC$.

כעת, נרצה להתבונן באלגוריתם הנאיבי ביותר לפתרון הבעיה.
- עבור על כל תת קבוצה של $k$ קודקודים ב$G$.
- לכל תת קבוצה $S\subseteq V$ כזו, עבור על כל הפרמוטציות האפשריות שלה. אם הקודקודים יוצרים מסלול פשוט השב 1.
- השב 0.

מהו זמן הריצה של אלגוריתם כזה? הרי מדובר ב:

$$\mathcal{O}\left(k!\cdot\binom{n}{k}\right)$$

זהו זמן ריצה שלא תואם לאלגוריתם FPT. לכן, גישה נאיבית לא תעבוד במקרה הזה. נתבונן במקום זאת ברעיון הבא:

**Algorithm: Color Coding for $k$-Path**

1. Color-set: $\{1, 2, \ldots, k\}$
2. Randomly assign a color to each vertex
3. Highlight a solution

כלומר, מעבר על כל צומת בגרף וצביעתו באופן ב"ת בשאר הצמתים בצבע מתוך $\{1, ..., k\}$. המטרה: אם אכן קיים מסלול כנ"ל, נקווה כי הצביעה האקראית תצבע את כל אחד מ$k$ הצמתים במסלול בצבעים שונים בסדר עולה מ1 עד $k$. כך למעשה, נוכל למצוא את אותו המסלול.

כעת נתבונן בניתוח הסתברותי של האלגוריתם.

נניח כי בגרף ישנו מסלול פשוט מאורך $k$, אזי נסמנו $(s_1, ...., s_k)$. לכל אחד מ$k$ הצמתים במסלול יש $k$ צבעים אפשריים לבחירה. לכן ההסתברות עבור כל צומת לצבע מסוים הינה $\frac{1}{k}$. לכן ההסתברות הכוללת לכך שאיטרציה בודדת תצליח לצבוע את המסלול הנכון בצורה טובה הינה:

$$\left(\frac{1}{k}\right)^k$$

אם כן, הסתברות זו נמוכה מאוד. לכן נרצה לחזור על התהליך כמה וכמה פעמים על מנת להגביר את ההסתברות להצלחה. לשם כך, נרצה לבצע כ$\mathcal{O}^*\left(k^k\right)$ איטרציות - בכל אחת מהן נצבע אקראית את הגרף. אם באיטרציה מסוימת נמצא מסלול כנ"ל לפי סדר הצביעה, סיימנו. אחרת נתקדם לאיטרציה הבאה. רק אם בסיום כל האיטרציות לא מצאנו אף מסלול כזה נחזיר שקר. נכתוב את האלגוריתם כך:

**Algorithm: Color Coding for $k$-Path (with Amplification)**

1. Repeat $T$ times (amplification: boosts the success probability)
   (a) Randomly assign each vertex a color from $\{1, 2, \ldots, k\}$
   (b) **Remove irrelevant edges:** for every edge connecting a vertex colored $i$ to a vertex not colored $i + 1$, discard the edge from the graph
   (c) **Check for a path:** let $R \leftarrow$ the set of vertices colored 1. For $i = 1, \ldots, k - 1$: update $R \leftarrow$ the set of vertices colored $i + 1$ that have an edge from some vertex in $R$
   (d) If, at the end, $R$ (the reachable vertices colored $k$) is non-empty: **return** "Yes"
2. If no iteration found a path: **return** "No" (correct with high probability)

כעת לניתוח ההצלחה והכישלון. אם כן, ההסתברות שאיטרציה בודדת תכשל הוא $1 - \frac{1}{k^k}$. כעת, נניח כי מבצעים $100k^k$ איטרציות בלתי תלויות באלגוריתם. אזי:

$$\Pr\left[\text{Algo failed}\right] = \prod_{i=1}^{100k^k}\left(1 - \frac{1}{k^k}\right) = \left(1 - \frac{1}{k^k}\right)^{100k^k} =_{n=k^k} \left(1 - \frac{1}{n}\right)^{100n} \approx e^{-100}$$

כלומר,

$$\Pr\left[\text{Sucess}\right] \geq 1 - \frac{1}{e^{100}}$$

ולכן האלגוריתם מצליח בהסתברות גבוהה מאוד. נשים לב כי אם האלגוריתם השיב "כן", אזי בהכרח קיים פתרון: המסלול שמצא וצבע.

נשים לב כי הגדרת מחלקת הסיבוכיות FPT לא מחייבת כי האלגו' יהיה דטרמיניסטי כלל, אלא רק שיהיה בזמן ריצתו מהצורה שתוארה. וגם רצינו הצלחה בהסתברות גבוהה. נשים לב כי זמן הריצה מורכב מ:

- חישוב בדיקת המסלול שניחש האלגוריתם. אלו $k$ צמתים, נניח בצורה הגיונית כי בדיקה כזו עלותה $\mathcal{O}(n)$: ע"י בדיקה במטריצת השכנויות.
- מס' האיטרציות - $k^k$.

נקבל כי זמן הריצה הינו:

$$\mathcal{O}\left(n \cdot k^k\right) = \mathcal{O}^*\left(k^k\right)$$

ולכן קיבלנו כי K-path $\in$ FPT.

הערה 5. בשנת 2010 מצאו אלגוריתם לבעיה בזמן ריצה $\mathcal{O}^*\left(1.657^k\right)$.

## 1.2 Branching: Bounded Search Trees - עצי חיפוש חסומים

כעת, מאלגוריתמים רנדומיים נעבור לדבר על אלגוריתמים דטרמיניסטיים פרמטריים. נזכר בבעיה Vertex Cover, בסגנון בעיית ההכרעה שלה:

$$\text{Vertex Cover} = \{(G,k) \,|\, \text{G has VC of size } \leq k\}$$

המטרה שלנו תהיה לבנות אלגוריתם בזמן $\mathcal{O}^*\left(2^k\right)$ - שהתלות האקספוננציאלית תהיה ב$k$ בלבד, ולא ב$n = |G|$.

**הגדרה 6.** אלגוריתמי עצי חיפוש מנוהלים לרוב באמצעות אוסף של חוקים מהצורה:

$$[\text{Condition}] : \text{Action}$$

וישנם כמה דגשים על אופן הפעלת החוקים:
- אנו תמיד סורקים את רשימת החוקים מההתחלה ומוציאים לפועל את החוק הראשון שהתנאי שלו מתקיים.
- אנו סורקים את רשימת התנאים מההתחלה כי לאחר שביצענו פעולה של חוק מסוים - חוק $X$, אזי הגרף השתנה. השינוי הזה יכול פתאום לגרום לכך שתנאי של חוק אחר שמופיע קודם ברשימה הפך להיות נכון. לכן, האלגוריתם יהיה חייב לחזור לסרוק מהתחלה את רשימת התנאים. לכן: הסדר של החוקים קריטי לנכונות האלגוריתם.

**הגדרה 7.** חוק צמצום (Reduction Rule). מדובר בפעולה שבה האלגוריתם מקבל החלטה דטרמיניסטית וברורה, וקורא לעצמו ברקורסיה לכל היותר פעם אחת. בחוק מסוג זה אין פיצול או ניחוש, אלא הקטנת הבעיה בלבד.

**הגדרה 8.** חוק פיצול (Branching Rule). פעולה בה האלגוריתם חייב לנחש או לפצל את הדרך, ולכן הוא קורא לעצמו ברקורסיה לפחות פעמיים. לדוגמה: או שנבחר את $u$ או שנבחר את $v$: זה ניחוש.

### 1.2.1 אלגוריתם לVC בזמן $\mathcal{O}^*\left(2^k\right)$

נתבונן באלגוריתם הבא:

**Algorithm:** $O^*(2^k)$**-Time Algorithm for Vertex Cover**

**ALG**($G$, $k$)**:**

1. **Rule 1.** If $G$ has no edges, **return** Yes
2. **Rule 2.** If $k = 0$, **return** No
3. **Rule 3.** Pick an edge $\{u, v\}$
   i. Call ALG$(G - u, k - 1)$
   ii. Call ALG$(G - v, k - 1)$
   (a) **Return** Yes iff at least one of the calls returns Yes

נתבונן באלגוריתם ונסבירו לעומק.

החוק הראשון - אם בגרף $G$ אין בכלל קשתות: נחזיר 1. מדוע? לפי הגדרת כיסוי קודקודים, קבוצת כיסוי קודקודים היא קבוצת קודקודים שמכסה את כל הקשתות בגרף. אם אין קשתות, ברור שיש קבוצת כיסוי קודקודים בגודל לכל היותר $k$, בפרט בגודל 0.

החוק השני - אם $k = 0$, נחזיר 0. מדוע? כי בגרף עדיין קשתות, לפי הסעיף הקודם שעברנו אותו ולכן הגענו לבדיקה זו, אך $k = 0$ ולכן התקציב שלנו לכיסוי נגמר. לכן אי אפשר לכסות את הקשתות הנותרות תחת המגבלה של $k$ קודקודים.

החוק השלישי - חוק הפיצול: אם הגענו לשלב זה, כלומר בגרף יש לפחות קשת אחת וגם $k > 0$. לכן:

בוחרים קשת כלשהי בגרף $\{u, v\}$. נשים לב כי בהכרח אחד מהקודקודים $u, v$ חייב להיות בכיסוי. לכן ישנן 2 אפשרויות:

- או ש$u$ בכיסוי, ואז נבצע רקורסיה ל$\mathrm{Alg}\left(G\backslash\{u\}, k-1\right)$.
- או ש$v$ בכיסוי, ואז נבצע רקורסיה ל$\mathrm{Alg}\left(G\backslash\{v\}, k-1\right)$.
- האלגוריתם יחזיר כן, אם ורק אם לפחות אחת מזוג הקריאות הרקורסיביות הנ"ל השיבו כן.

הערה 9. הטכניקה הנ"ל נקראת על שם עצי חיפוש, כיוון שהאלגוריתם מייצר מבנה נתונים מרחבי של עץ במהלך הריצה הרקורסיבית שלו.

**משפט 10.** ( הוכחת נכונות האלגוריתם ) האלגוריתם הנ"ל מחזיר 1 $\iff$ קיים ב$G$ כיסוי קודקודים מגודל $k$ לכל היותר.

הוכחה: נוכיח באינדוקציה שלמה כפולה על מס' הצמתים $n$ ועל $k$.

בסיס: אם $k = 0$, אז קיימת לפחות קשת אחת האלגוריתם מחזיר 0 בצדק כי בלתי אפשרי לכסות את הקשתות הללו מבלי לבחור קודקודים. אם בגרף אין קשתות בכלל, כל קבוצת קודקודים ריקה $S = \emptyset$ מהווה כיסוי קודקודים חוקי בגודל $k \leq 0$ ולכן האלגוריתם מחזיר בצדק 1.

צעד: נניח שהאלגוריתם פועל נכון ומחזיר תשובה נכונה, עבור קלט בו מס' הצמתים קטן ממש מ$n$. נוכיח שהאלגוריתם פועל נכון עבור גרף עם $n$ צמתים ותקציב $k$: תחת ההנחה כי $k > 0$. כלומר: חוקים $1, 2$ אכן לא מתקיימים. אם כן, כיוון שחוקים 1 ו2 לא מתקיימים אזי האלגו' בוחר קשת כלשהי $\{u, v\}$ ומבצע פיצול לזוג קריאות:

$$Alg\left(G\backslash\{u\}, k-1\right), Alg\left(G\backslash\{v\}, k-1\right)$$

כעת נוכיח שקילות בשני הכיוונים.

בכיוון ראשון,

טענה: אם $(G, k)$ מכיל פתרון: כיסוי $S$ חוקי המקיים $|S| \leq k$, אזי לפחות אחת משתי הקריאות תחזיר כן.

הוכחה: $\{u, v\}$ היא קשת חוקית ב$G$. הקבוצה $S$ מכסה את כל הקשתות בגרף, ולכן בהכרח לפחות אחד משני קצוותיה של הקשת $u, v$ חייב להמצא ב$S$.

אם $u \in S$, נגדיר $S' = S\backslash\{u\}$ . קבוצה זו היא כיסוי קודקודים לגרף המוקטן $G\backslash\{u\}$ שכן הוא לא כולל את הקודקוד הנ"ל, והיא בגודל $k - 1$ לכל היותר. מהנחת האינדוקציה, יתקיים $Alg\left(G\backslash\{u\}, k-1\right) = 1$: שכן קיים כיסוי בגודל לכל היותר $k - 1$ לתת הגרף.

אם $v \in S$, בדומה עם $S' = S\backslash\{v\}$.

לכן האלגוריתם מחזיר 1 כי מוחזר 1 בלפחות אחת מהקריאות.

בכיוון שני,

טענה: אם האלגוריתם מחזיר 1, אזי לזוג $(G, k)$ אכן יש פתרון חוקי. כלומר, קיים כיסוי בגודל לכל היותר $k$.

הוכחה: נניח כי האלגוריתם החזיר 1. כלומר, אחת מהקריאות הרקורסיביות החזירה 1. בה"כ, נניח כי הקריאה הרקורסיבית על תת הבעיה הראשונה $Alg\left(G\backslash\{u\}, k-1\right)$ החזירה 1. זו קריאה שרצה על תת הגרף $G\backslash\{u\}$ ולכן לו קיים כיסוי קודקודים מגודל $k - 1$ לכל היותר. נסמנו $S$. נגדיר את הכיסוי:

$$S_T = S \cup \{u\}$$

קל לראות שמדובר בכיסוי ל$G$. כנדרש.

**משפט 11.** זמן הריצה של האלגוריתם הוא $\mathcal{O}^*\left(2^k\right)$ ולכן מדובר באלגוריתם מסוג $FPT$.
הוכחה: יהי $N\left(k\right)$ מס' הקריאות הרקורסיביות בעלים של עץ החיפוש. הרי כי אם נסתכל על הנוסחה כתלות בפרמטר $k$, שהרי היא תלויה בו ולא בגודל הגרף נקבל כי:

$$N\left(k\right)=2N\left(k-1\right),N\left(0\right)=1$$

הרי כי:

$$N\left(k\right)=2^iN\left(k-i\right)$$

$$k=i\Longrightarrow N\left(k\right)=2^k\cdot N\left(0\right)=2^k$$

לכן מס' הקריאות הרקורסיביות הוא $2^k$, בכל קריאה רקורסיבית האלגוריתם צריך לעבור על כל הגרף - לעדכן קשתות וכו'. בזמן ריצה של $\mathcal{O}\left(n+m\right)$. לכן סה"כ זמן הריצה

$$\mathcal{O}\left(2^k\cdot\left(n+m\right)\right)=\mathcal{O}^*\left(2^k\right)$$

כנדרש.

הערה 12. נשים לב מדוע $N\left(k\right)$, כלומר מדוע מס' האיטרציות תלוי ב$k$: הצלחנו לתאר את הבעיה כאחריות ב$k$, כלומר - האלגוריתם שתואר עובד כתנאי עצירה רק כש$k=0$, לכן ללא תלות בגודל הגרף $G$, מס' האיטרציות תלוי ב$k$.

## 1.3 Kernelization

**הגדרה 13.** בהינתן מופע של בעיה ותקציב $(\Pi,k)$, אלגוריתם גרעיון (Kernelization) מייצר בזמן פולינומי מופע $(\Pi',k')$ שקול של אותה בעיה כך שמתקיים:
1. גודל המופע החדש חסום בפונקציה של $k$: $|\Pi'|\leq f\left(k\right)$
2. התקציב החדש לא גדל: $k'\leq k$.

הרעיון של Kernelization, הוא לנקות ולצמצם את החלקים הטריוואלים והמיותרים של הקלט עוד לפני שאנו מתאמצים לפתור אותו. כך, נשאר עם "גרעין" שגודלו תלוי ב$k$ בלבד.

**דוגמה 14.** Kernelization עבור VC:
נתבונן באוסף חוקים, הגיוניים מאוד ביחס להגדרת הבעיה VC -
1. Rule 1: אם קיים בגרף צומת מבודד, הסר אותו.
כלל זה הגיוני שכן צומת מבודד לא נוגע באף קשת, לכן אין סיבה שיבחר לכיסוי הקודקודים.
2. Rule 2: אם קיים צומת בגרף עם לפחות $k+1$ שכנים, הסר אותו מהגרף ועדכן $k=k-1$.
אם לא נקח את הצומת הנ"ל לתוך כיסוי הקודקודים, בהכרח נצטרך לקחת את כל $k+1$ השכנים שלו על מנת לכסות את הקשתות בינם לבינו. אך מותר לנו לקחת $k$ בלבד לכל היותר, ולכן זה לא יתכן. לכן בהכרח הוא יבחר, לכן טריוואלית ניתן מראש להסירו מהגרף ולהוריד את $k$.
3. Rule 3: נפעיל את חוקים $1,2$ עד שלא ניתן יותר נבדוק בסיום כמה קשתות נשארו בגרף. אם הגרף עדיין מכיל יותר מ$k^2$ קשתות: השב $0$

חוק 2 דאג כי לכל צומת ישארו לכל היותר $k$ שכנים. הרי, אם בגרף ישנם יותר מ$k^2$ קשתות - בהכרח לא קיים שום כיסוי בגודל $k$ שמכסה את כולם.

המסקנה שלנו מתהליך הKernelization: נשארנו עם גרעין בו לכל היותר $k^2$ קשתות, ובעקבות כך: $|n'| \leq 2k^2$ צמתים. הגענו למופע קטן ומצומצם שגודלו תלוי אך ורק ב$k$, ועליו כעת ניתן להריץ איזה אלגוריתם שנרצה ביעילות עצומה. כעת, נוכל להריץ את האלגו' הקודם שראינו לVC בזמן $\mathcal{O}^*\left(2^k\right)$.

נרצה לנתח את זמן הריצה של הקרנול - זמן זה הוא לכל היותר $\mathcal{O}\left(n+m+k^3\right)$, באשר ה$k^3$ מגיע מהבדיקות והטיפול בשכנים וכו'. כעת, נרצה להריץ אלגו' חיפוש על הגרעין:

$$n'=k^2, m'=k^2 \Longrightarrow 2^k \cdot (n'+m') = 2^k\left(k^2+k^2\right) = \mathcal{O}\left(2^{k+1}\cdot k^2\right)$$

**דוגמה 15.** אם $k=40, n=10000, m=30000$ נקבל:
- בשיטה הנאיבית, שראינו בהתחלה: $\sim 4.53 \cdot 10^{116}$ פעולות.
- בBranching לבד: $\sim 4.4 \cdot 10^{16}$
- בשיטת הגרעון: $\sim 3.5 \cdot 10^{15}$

כלומר, אמנם אין שינוי באסימפטוטיות אך מבחינת מס' פעולות תהליך הגרעון תרם מאוד לריצת האלגוריתם.

## 1.4 Closest String - המחרוזת הקרובה ביותר

כעת נתאר בעיה מרכזית במדעי המחשב, מציאת מחרוזת מרכזית כך שהמרחק שלה מכל שאר המחרוזות קטן או שווה מתקציב מסוים. נפרמלה:

**הגדרה 16.** בהינתן זוג מחרוזות באותו אורך: $S_1 = s_1^1, ....., s_1^p, S_2 = s_2^1, ....., s_2^p$ נגדיר את מרחק האמינג להיות גודל קבוצת האינדקסים בהם התווים שונים:

$$\text{Ham}\left(S_1, S_2\right) = |\left\{i \in \{1, ....., p\} : s_1^i \neq s_2^i\right\}|$$

**הגדרה 17.** בעיית הCloset string מוגדרת באופן הבא: בהינתן קבוצה של $t$ מחרוזות $\left\{S^1, ...., S^t\right\}$ באורך $p$ כל אחת, ופרמטר $d \geq 0$ נרצה להחזיר 1 אם ורק אם קיימת מחרוזת $S^*$ באורך $p$ כך שמרחק האמינג בינה לבין כל מחרוזת אחרת הוא לכל היותר $d$. כלומר:

$$\text{Closet-String} = \left\{\left(S^1, ...., S^t, d\right) | \exists S^* \text{ s.t } |S^*| = p \text{ and } \forall i \in [t] : \text{Ham}\left(S^*, S^i\right) \leq d\right\}$$

ראשית, אם נרצה להתייחס לp כפרמטר שלנו: נוכל לבדוק את כל האפשרויות לפונקציית המטרה $S^*$: $2^p$ בדיוק. אך, אורך המחרוזת $p$ יכול להיות גדול מאוד - ולכן פרמטר כזה לא יתן לנו FPT יעיל כי תלוי באורך הקלט ולא בתקציב השגיאה. לכן, נרצה להתייחס לd כפרמטר שלנו. נעיר כי אם $d > p$ אזי ברור שנחזיר ישר כן, כי ניתן לחרוג מכל אורך המחרוזת. לכן למשל המחרוזת $S^* = 0000...000$ תעבוד.

נרצה לשאול את עצמנו שאלה לגיטמית - האם המחרוזת $S^*$ יכולה להיות כל מחרוזת בעולם? לא. אם נחפש במרחב האינסופי, לא נגיע לזמן ריצה רצוי של FPT. לכן נגיע לתובנה הבאה: המחרוזת $S^*$ חייבת להיות "קרובה" למחרוזות הנתונות. כלומר, היא צריכה להראות כמו אחת המחרוזות הנתונות בצירוף של כמה תיקונים. לשם כך, נגדיר בעיה חדשה.

**הגדרה 18.** בעיית Closet string מוגדרת באופן הבא: בהינתן קבוצה של $t$ מחרוזות $\{S^1, ...., S^t\}$ באורך $p$ כל אחת, ופרמטר $d \geq 0$ וכן פרמטר $\Delta \geq 0$, נרצה לבדוק אם אנחנו יכולים לבצע לכל היותר $\Delta$ תיקונים ( שינוי של ביטים במחרוזת ) הראשונה $S^1$ ( בה"כ) כדי לקבל ממנה מחרוזת חדשה שתשמש כ$S^*$. כך שיתקיים:

$$\forall i \in \{2, ..., t\} : \text{Ham}\left(S^*, S^i\right) \leq d$$

הערה 19. זוג הגרסאות של הבעיה הן NPC.

כעת נציג אלגוריתם מבוסס חוקים שפותר את הבעיה:

**Algorithm: $O^*((d+1)^d)$-Time Algorithm for Closest String**

**ALG($S^1, \ldots, S^t$, $\Delta$):**

1. **Rule 1.** If $\text{Ham}(S^1, S^i) \leq d$ for all $2 \leq i \leq t$, **return** Yes
2. **Rule 2.** If $\Delta = 0$, **return** No
3. **Rule 3.** Let $S^i$ be such that $\text{Ham}(S^1, S^i) > d$. Let $P$ be a set of exactly $d+1$ positions where $S^1$ and $S^i$ differ
   - For every position $q$ in $P$, call $\text{ALG}(S^{1\prime}, S^2, \ldots, S^t, \Delta - 1)$, where $S^{1\prime}$ equals $S^1$ with the $q$-th letter being the $q$-th letter of $S^i$
4. **Return** Yes iff at least one of the calls returns Yes

נעבור על תיאור האלגוריתם. מדובר באלגוריתם רקורסיבי, שמקבל כקלט את המחרוזות ואת תקציב השינויים הנוכחי $\Delta$.
Rule 1־ אם מרחק האמינג בין המחרוזת הנוכחית לבין כל שאר המחרוזות הוא לכל היותר $d$:

$$\forall i \in \{2, ..., t\} : \text{HAM}\left(S^1, S^i\right) \leq d$$

אזי מצאנו מחרוזת מתאימה, ונחזיר 1.
Rule 2־ זהו בסיס הכשלון. אם נגמר התקציב, וגם אנחנו כאן כי שלב 1 נכשל, אזי החזר 0.
Rule 3־
* נמצא מחרוזת $S^i$ כך ש$\text{Ham}\left(S^1, S^i\right) \geq d+1$.
* נגדיר את הקבוצה $P$ להיות קבוצה של בדיוק $d+1$ אינדקסים שונים מתוך כלל האינדקסים שבהם המחרוזת $S^1$ ו$S^i$ שונות.

נשים לב: מדוע שנבחר קבוצה בגודל $d+1$ בדיוק ולא של כל האינדקסים השונים? יתכן כי מס' האינדקסים השונים בלתי מוגבל, ובפרט הוא $p$ למשל. אם האלגוריתם היה פותח ענף רקורסיבי לכל אחד מהם, התלות לא הייתה ב$d$ כבר. וכן, הוכח ולא נוכיח כי מספיק לקחת קבוצה בגודל $d+1$ של האינדקסים השונים.

* לכל אינדקס $q$ בתוך $P$, ניצור מופע חדש בו נתקן את האות ב$S^1$ להיות כמו ב$S^i$, נקטין את התקציב ל$\Delta - 1$ ונפעיל את האלגוריתם מחדש. נחזיר 1 אם ורק אם אחת הקריאות החזירה 1.

**משפט 20.** הוכחת נכונות האלגוריתם.
הוכחה:
בכיוון ראשון, נניח כי קיים פתרון תקין $S^*$. אם המצב הנוכחי לא מקיים עדיין את $Rule\ 1$, חייבת להיות לפחות מחרוזת אחת $S^i$ המקיימת $Ham\left(S^1, S^i\right) > d$. כיוון ש$S^*$ ומצאת במרחק של לכל היותר $d$ גם מ$S^1$ וגם מ$S^i$, לפי אי שוויון המשולש לפחות אחד מהשינויים הנדרשים כדי להפוך את $S^1$ ל$S^*$ חייב להיות באחד מ$d+1$ האינדקסים בקבוצה $P$ - נשים לב שכאן במשפט זה עומדת כל הנכונות של האלגוריתם: שכן אם נניח בשלילה ש$S^*$ לא נבדלת מ$S^1$ באף אחד מהאינדקסים בקבוצה, יתקיים $Ham\left(S^*, S^i\right) \geq d+1$ בסתירה, אינטואיטיבית זה נכון כי אם ישנם $d+1$ הבדלים, צריך לתקן לפחות אחד מהם בהכרח. לכן, אחת מהקריאות הרקורסיביות תבצע בדיוק את התיקון הנכון , תקטין את $\Delta$ ותמשיך באינדוקציה לפי הנכונות. האלגו' סורק את כל $d+1$ האפשרויות בקבוצה, ומחזיר $1$ אפ"מ אחת האפשרויות החזירה $1$, כנדרש.
בכיוון שני, נניח שהאלגו' החזיר $1$. לפי מבנה האלגוריתם תשובה חיובית מוחזרת אפ"מ התקיים תנאי $1$, או שזוהתה שרשרת תיקונים חוקית בעלות חוקית שמובילה לשם. לכן קיימת מחרוזת $S^*$ כנ"ל.

**משפט 21.** זמן הריצה של האלגוריתם הוא $\mathcal{O}^*\left((d+1)^d\right)$ ולכן מזובר באלגוריתם מסוג $FPT$.
הוכחה: יהי $N(\Delta)$ מס' הקריאות הרקורסיביות. ברור כי:

$$N(\Delta) = (d+1)\,N(\Delta-1), N(0) = 1 \Longrightarrow N(d) = (d+1)^d$$

כלומר, אנו נניח כי $\Delta = d$, מס' השינויים שניתן לבצע לכל היותר במחרוזת הראשונה הוא בדיוק $d$. זה מס' הרמות של העץ ברקורסיה וזהו משתנה שמחזיק ערך של פרמטר לפיו אנו מודדים את זמן הריצה. נשים לב כי בכל שלב אנו מבצעים פעולות פולינומיאליות בגודל הקלט כמו בדיקת מרחק האמינג ל$t$ מחרוזות, לכן סה"כ זמן הריצה הינו:

$$\mathcal{O}^*\left((d+1)^d\right)$$

0<0